

计算机组成原理

王鸿鹏

计算机科学与技术学院

wanghp@hit.edu.cn

计算机的运算方法

- 算术移位与逻辑移位
- 定点运算
 - 加减法运算
 - 一位乘法运算
 - booth算法
 - 除法运算
- 浮点运算

移位运算

- 移位的意义

$$11.m = 11\underline{00}.cm$$

小数点右移 2 位

机器用语

11 相对于小数点 左移 2 位

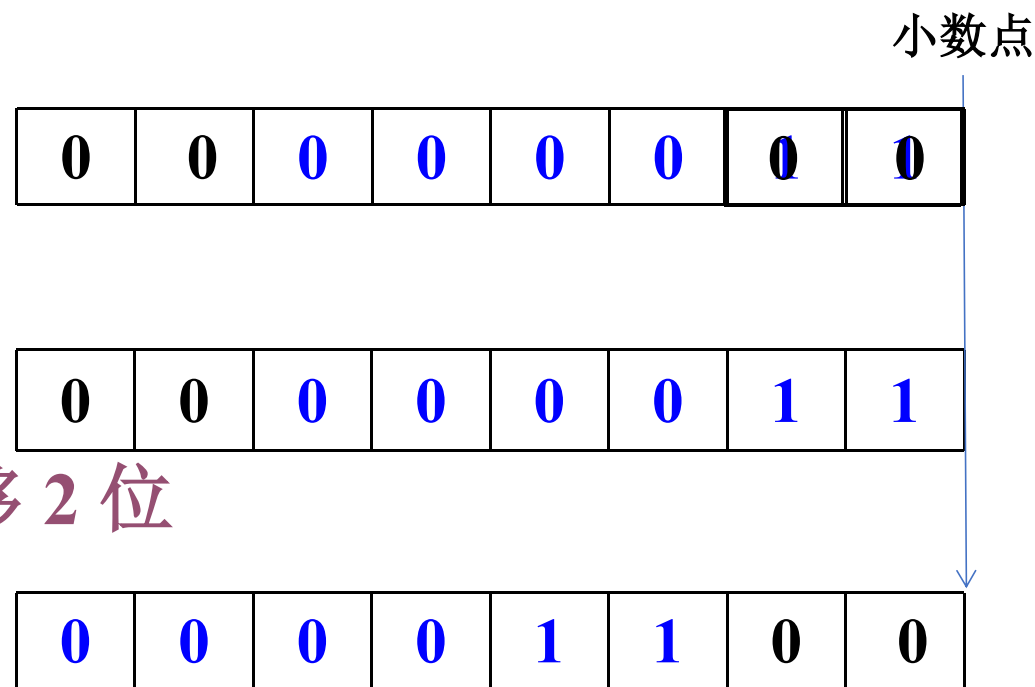
(小数点不动)

左移

绝对值扩大

右移

绝对值缩小



- 在计算机中，移位与加减阶码配合，能够实现乘除运算

算术移位规则

- 符号位不变
- 机器数的移位 与 真值移位 一致

真值	机器数表示	空位补什么？
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移 补 0
		右移 补 1
	反 码	1

$$y = 0.y_1y_2 \dots y_n 100 \dots 00$$

$$y = -0.y_1y_2 \dots y_n 100 \dots 00$$

$$[y]_{\text{原}} = 1.y_1y_2 \dots y_n 100 \dots 00$$

$$\text{右移1位: } 1.\mathbf{0}y_1y_2 \dots y_n 100 \dots 0$$

$$\text{左移1位: } 1.y_2y_3 \dots y_n 100 \dots 00\mathbf{0}$$

$$[y]_{\text{反}} = 1.\overline{y}_1\overline{y}_2 \dots \overline{y}_n 011 \dots 11$$

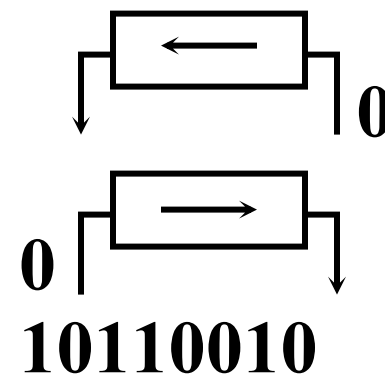
$$\begin{aligned}
 [y]_{\text{补}} &= [y]_{\text{反}} \text{ 的末位} + \mathbf{1} \\
 &= 1.\overline{y}_1\overline{y}_2 \dots \overline{y}_n 100 \dots 00
 \end{aligned}$$

算术移位和逻辑移位的区别

- 算术移位：有符号数的移位
- 逻辑移位：无符号数的移位

逻辑左移 低位补 0，高位丢弃

逻辑右移 高位补 0，低位丢弃



例：补码表示的机器数 10110010

逻辑左移 0110010**0**

逻辑右移 **0**1011001

算术左移 1110010**0**

算术右移 1**1**011001

正整数移位运算例题

- 设机器数字长为 8 位（含 1 位符号位），写出 $A = +26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A = +26 = +11010$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0,0011010$

移位操作	机 器 数	对应的真值
	$[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}}$	
无	0,0011010	+26
左移一位	0,011010 0	+52
左移两位	0,11010 00	+104
右移一位	0, 0 001101	+13
右移两位	0, 00 00110	+6

负整数移位运算例题

- 设机器数字长为 8 位（含 1 位符号位），写出 $A = -26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

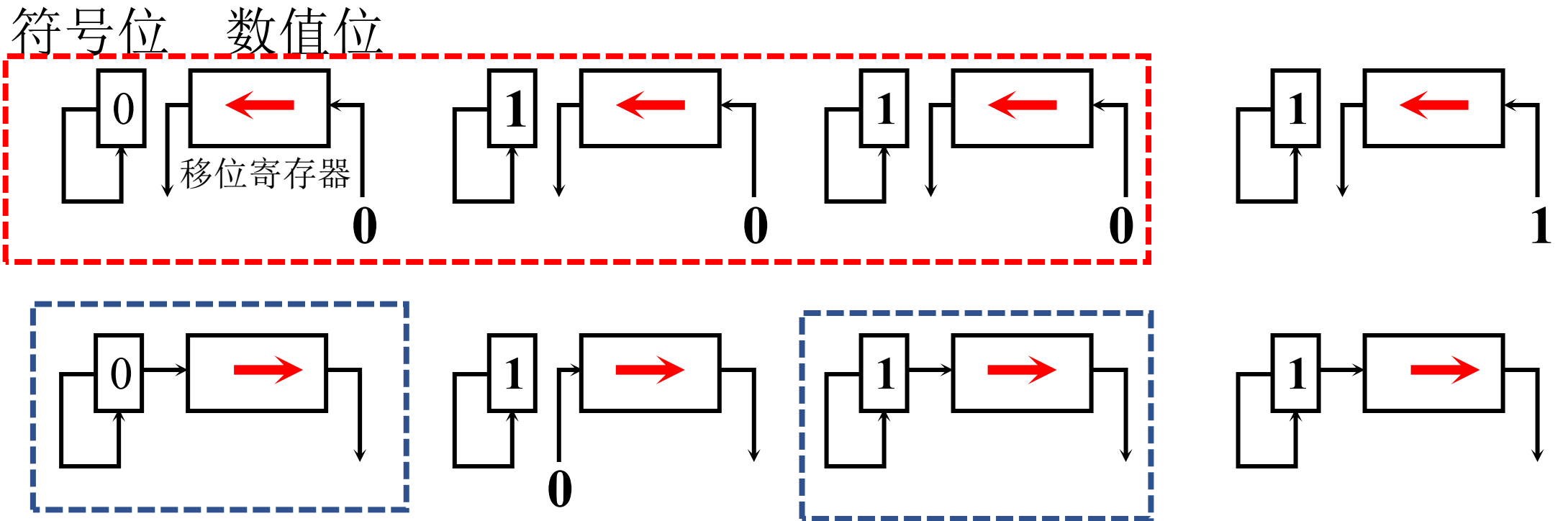
解： $A = -26 = -11010$

原码	移位操作	机 器 数	对应的真值
	移位前	1,0011010	- 26
	左移一位	1,0110100	- 52
	左移两位	1,1101000	- 104
	右移一位	1,0001101	- 13
	右移两位	1,0000110	- 6

补码移位运算示例

移位操作	机 器 数	对应的真值
给定数	1,1100110	– 26
左移一位	1,1001100	– 52
左移两位	1,0011000	– 104
右移一位	1,1110011	– 13
右移两位	1,1111001	– 7

算术移位的硬件实现



a 真值为正
← 丢 1 出错
→ 丢 1 影响精度

b 负数的原码
出错
影响精度

c 负数的补码
正确
影响精度

d 负数的反码
正确
正确

计算机的运算方法

- 算术移位与逻辑移位
- 定点运算
 - 加减法运算
 - 一位乘法运算
 - booth算法
 - 除法运算
- 浮点运算

定点加减法运算（补码）

●加法

整数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \quad (\text{mod } 2^{n+1})$

小数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \quad (\text{mod } 2)$

●减法

$$A - B = A + (-B)$$

整数 $[A - B]_{\text{补}} = [A + (-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \quad (\text{mod } 2^{n+1})$

小数 $[A - B]_{\text{补}} = [A + (-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \quad (\text{mod } 2)$

连同符号位一起相加，符号位产生的进位自然丢掉

例：用补码运算求 $A+B$

- 设 $A = -9$, $B = -5$ （设A和B数值位数为4），**用补码运算求 $A+B$**

解： $[A]_{\text{补}} = 1, 0111$

$+ [B]_{\text{补}} = 1, 1011$

$$[A]_{\text{补}} + [B]_{\text{补}} = \textcolor{red}{1}1, 0010 = [A + B]_{\text{补}}$$

$\therefore A + B = -1110$

验证

$$\begin{array}{r} -1001 \\ + -0101 \\ \hline -1110 \end{array}$$

已知 $[X]_{\text{补}}$ ，以下哪个方法可以求出 $[-X]_{\text{补}}$ ？

- ☐ A 包括符号位在内按位取反
- ☒ B 包括符号位在内，每位取反，末位加 1
- ☐ C 除符号位外，每位取反，末位加 1
- ☐ D 符号位取反，其他保持不变

提交

关于 $[X]_{\text{补}}$ 和 $[-X]_{\text{补}}$ ，哪个表述正确？

- ☒ A $[-X]_{\text{补}} + [X]_{\text{补}} = 0$
- ☐ B $[X]_{\text{补}} = [-X]_{\text{补}}$
- ☐ C $[X]_{\text{补}}$ 和 $[-X]_{\text{补}}$ 没有任何关系

提交

*例：已知整数 $[X]_{\text{补}}$ ，求 $[-X]_{\text{补}}$

设 $[X]_{\text{补}} = X_n, X_{n-1} \dots X_1 X_0$ ，根据符号位 X_n 为0或1分开讨论

<I>

$$[X]_{\text{补}} = 0, X_{n-1} X_{n-2} \dots X_1 X_0$$

$$X_{\text{原}} = 0, X_{n-1} X_{n-2} \dots X_1 X_0$$

$$[-X]_{\text{原}} = 1, X_{n-1} X_{n-2} \dots X_1 X_0$$

$$[-X]_{\text{补}} = 1, (\bar{X}_{n-1} \bar{X}_{n-2} \dots \bar{X}_1 \bar{X}_0 + 1)$$

<II>

$$[X]_{\text{补}} = 1, X_{n-1} X_{n-2} \dots X_1 X_0$$

$$X_{\text{原}} = 1, (\bar{X}_{n-1} \bar{X}_{n-2} \dots \bar{X}_1 \bar{X}_0 + 1)$$

$$[-X]_{\text{原}} = 0, (\bar{X}_{n-1} \bar{X}_{n-2} \dots \bar{X}_1 \bar{X}_0 + 1)$$

$$[-X]_{\text{补}} = 0, (\bar{X}_{n-1} \bar{X}_{n-2} \dots \bar{X}_1 \bar{X}_0 + 1)$$

$[X]_{\text{补}}$ 连同符号位在内，每位取反，末位加1，即得 $[-X]_{\text{补}}$

观察发现 $[-X]_{\text{补}} + [X]_{\text{补}} = 2^{n+1} \rightarrow 0 \pmod{2^{n+1}}$

*例：已知小数 $[y]_{\text{补}}$ ，求 $[-y]_{\text{补}}$

设 $[y]_{\text{补}} = y_0.y_1y_2 \dots y_n$ ，根据符号位 y_0 为0或1分开讨论

<I> $[y]_{\text{补}} = 0.y_1y_2 \dots y_n$

$$[y]_{\text{原}} = 0.y_1y_2 \dots y_n$$

$$-y = \text{---} 0.y_1y_2 \dots y_n$$

$$[-y]_{\text{补}} = \textcolor{red}{1} + (0.\overline{y_1}\overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$[-y]_{\text{补}} = \textcolor{red}{1}.\overline{y_1}\overline{y_2} \dots \overline{y_n} + 2^{-n}$$

<II> $[y]_{\text{补}} = 1.y_1y_2 \dots y_n$

$$[y]_{\text{原}} = \textcolor{red}{1} + (0.\overline{y_1}\overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$y = \text{---} (0.\overline{y_1}\overline{y_2} \dots \overline{y_n} + 2^{-n})$$

$$-y = 0.\overline{y_1}\overline{y_2} \dots \overline{y_n} + 2^{-n}$$

$$[-y]_{\text{补}} = 0.\overline{y_1}\overline{y_2} \dots \overline{y_n} + 2^{-n}$$

$[y]_{\text{补}}$ **连同符号位**在内，每位取反，末位加 1，即得 $[-y]_{\text{补}}$

观察发现 $[-y]_{\text{补}} + [y]_{\text{补}} = 2 \text{ ---} \textcolor{red}{0} \pmod{2}$

例：用补码运算求 $A-B$

设机器数字长为 8 位（含 1 位符号位）且 $A = 15$, $B = 24$, 用补码求 $A - B$ 。

$$\text{解: } A = 15 = 0001111$$

$$B = 24 = 0011000$$

$$[A]_{\text{补}} = 0, 0001111 \quad [B]_{\text{补}} = 0, 0011000$$

$$+[-B]_{\text{补}} = 1, 1101000$$

$$[A]_{\text{补}} + [-B]_{\text{补}} = 1, 1110111 = [A - B]_{\text{补}}$$

$$\therefore A - B = -1001 = -9$$

设机器数字长为 8 位（含 1 位符号位）且 $A = -97$ ， $B = 41$ ，用补码求 $A - B$ （8 位补码表示）。（ $97 = 01100001$ $41 = 00101001$ ）

- ☐ A 11110110
- ☐ B 01110110
- ☐ C 11001000
- ☐ D 其他

例：用补码运算求 $A-B$

- 设机器数字长为 8 位（含 1 位符号位）且 $A = -97$, $B = 41$, 用补码求 $A - B$ 。（ $97 = 01100001$ $41 = 00101001$ ）

解： $A = -97 = -1100001$

$$[A]_{\text{补}} = 1, 0011111$$

$$+[-B]_{\text{补}} = 1, 1010111$$

$$[A]_{\text{补}} + [-B]_{\text{补}} = \mathbf{10}, 1110110 = [A - B]_{\text{补}}$$

$$\therefore A - B = +118 \quad (\text{溢出})$$

一位符号位判溢出

- 参加**补码加法运算**的两个数（减法时+减数相反数的补码）
 - 如果符号不同，不会溢出。
 - 如果符号相同（同0或同1），运算结果的符号与原操作数的符号不同，即为溢出
- 最高数值有效位的进位 $\oplus$ 符号位的进位 = 1，溢出
- 最高数值有效位的进位 $\oplus$ 符号位的进位 = 0，无溢出
- 硬件实现——异或门

两位符号位判溢出

- 小数变形补码

$$[x]_{\text{补}'} = \begin{cases} x & 1 > x \geq 0 \\ 4 + x & 0 > x \geq -1 \end{cases} \quad (\text{mod } 4)$$

- 整数变形补码

$$[X]_{\text{补}'} = \begin{cases} X & 2^n > X \geq 0 \\ 2^{n+2} + X & 0 > x \geq -2^n \end{cases} \quad (\text{mod } 2^{n+2})$$

- 最高符号位 代表其 真正的符号

- 结果的双符号位 相同 未溢出
- | | | | |
|-----|--------|-----|--------|
| 00, | xxxxxx | 00. | xxxxxx |
| 11, | xxxxxx | 11. | xxxxxx |

- 结果的双符号位 不同 溢出
- | | | | |
|-----|--------|-----|--------|
| 10, | xxxxxx | 10. | xxxxxx |
| 01, | xxxxxx | 01. | xxxxxx |

双符号位判断补码运算溢出

- 设机器数字长为 **9** 位（含 2 位符号位）且 $A = -97$, $B = 41$, 用补码求 $A - B$ 。 ($97 = 01100001$ $41 = 00101001$)

解: $A = -97 = -1100001$

$$B = 41 = 0101001$$

$$[A]_{\text{补}} = 11, 0011111$$

$$+ [-B]_{\text{补}} = 11, 1010111$$

$$[A]_{\text{补}} + [-B]_{\text{补}} = 1\mathbf{10}, 1110110 = [A - B]_{\text{补}}$$

$$\therefore A - B = -10 \quad (\text{溢出导致结果错误})$$

溢出的实例验证

	补码 ($n=8$)	十进制数	补码 ($n=8$)	十进制数
127-1	00000000	0	10000000	-128
-128+1	00000001	1	10000001	-127
	00000010	2	10000010	-126
64+63	...	...	...	...
	00111111	63	10111111	-65
64+64	01000000	64	11000000	-64
	01000001	65	11000001	-63
-64-64	...	...	...	...
	01111101	125	11111101	-3
-64-65	01111110	126	11111110	-2
	01111111	127	11111111	-1

计算机的运算方法

- 算术移位与逻辑移位
- 定点运算
 - 加减法运算
 - 一位乘法
 - booth算法
 - 除法运算
- 浮点运算

乘法运算——笔算乘法分析(选学)

$$A = -0.1101 \quad B = 0.1011$$

$$A \times B = -0.10001111$$

乘积的符号心算求得

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 0.10001111 \end{array}$$

✓ 符号位单独处理

✓ 乘数的某一位决定是否加被乘数

? 4位的积一起相加

✓ 乘积的位数扩大一倍

笔算乘法的改进（小数）（选学）

$$A \cdot B = A \cdot 0.1011$$

$$= 0.1A + 0.00A + \underline{0.001A} + 0.0001A$$

$$= 0.1A + 0.00A + \underline{0.001}(1 \cdot A + 0.1A)$$

$$= 0.1A + 0.01[0 \cdot A + 0.1(1 \cdot A + 0.1A)]$$

$$= 0.1\{1 \cdot A + 0.1[0 \cdot A + 0.1(1 \cdot A + \underline{0.1A})]\}$$

右移一位

$$= \underline{2^{-1}}\{1 \cdot A + 2^{-1}[0 \cdot A + 2^{-1}(1 \cdot A + \underline{2^{-1}(1 \cdot A + 0)})]\}$$

1 被乘数 $A + 0$

2 右移一位，得新的部分积

3 部分积 + 被乘数

...

右移一位，得结果

①

②

③

⑧

改进后的笔算乘法过程（小数，竖式）（选学）

0.1101
× 0.1011

?.????????

0.1101
× 0.1011

1101
+ 1101

100111
+ 0000

100111
+ 1101

0.10001111

部分积(乘数暂存)	丢弃	说明
0.0000	1011	初态，部分积高位=0
+ 0.1101		乘数为 1，加被乘数
0.1101	1011	→ 1，（逻辑右移）
0.0110	1101	形成新的部分积
+ 0.1101		乘数为 1，加被乘数
1.0011	1101	→ 1，
0.1001	1110	形成新的部分积
+ 0.0000		乘数为 0，加 0
0.1001	1110	→ 1，
0.0100	1111	形成新的部分积
+ 0.1101		乘数为 1，加被乘数
1.0001	111	→ 1，
0.1000	1111	无乘数位，结束

乘法运算——笔算乘法分析（整数）

$$A = -1101 \quad B = +1011$$

$$A \times B = -10001111$$

乘积的符号心算求得

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 10001111 \end{array}$$

- ✓ 符号位单独处理
- ✓ 乘数的某一位决定是否加被乘数
- ? 4位的积一起相加
- ✓ 乘积的位数扩大一倍

笔算乘法的改进

$$A \cdot B = A \cdot 1011 = 2^n \cdot A \cdot 0.1011 \quad (n=4)$$

$$= 2^n \cdot (0.1A + 0.00A + \underline{0.001A} + 0.0001A)$$

$$= 2^n \cdot (0.1A + 0.00A + \underline{0.001(1 \cdot A + 0.1A)})$$

$$= 2^n \cdot (0.1A + \underline{0.01[0 \cdot A + 0.1(1 \cdot A + 0.1A)]})$$

$$= 2^n \cdot (\underline{0.1}\{1 \cdot A + 0.1[0 \cdot A + 0.1(1 \cdot A + \underline{0.1A})]\})$$

$$= 2^n \cdot (\underline{2^{-1}}\{1 \cdot A + 2^{-1}[0 \cdot A + 2^{-1}(1 \cdot A + \underline{2^{-1}(1 \cdot A + 0)})]\})$$

右移一位

1 被乘数 $A + 0$

2 右移 1 位, 得新的部分积

3 部分积 + 被乘数

...

右移 1 位, 得结果

①

②

③

⑧

改进后的笔算乘法过程（整数，竖式）

	部分积(乘数暂存)		丢弃	说明
1101 × 1011 ----- ????????	0	0000	101 <u>1</u>	初态，部分积高位=0
	+ 0	1101		乘数为 <u>1</u> ，加被乘数
	0	1101	101 <u>1</u>	→1，（逻辑右移）
	0	0110	1 <u>101</u>	形成新的部分积
	+ 0	1101		乘数为 <u>1</u> ，加被乘数
	1	0011	1 <u>101</u>	→1，
	0	1001	11 <u>10</u>	形成新的部分积
	+ 0	0000		乘数为 <u>0</u> ，加 0
	0	1001	11 <u>10</u>	→1，
	0	0100	11 <u>1</u>	形成新的部分积
	+ 0	1101		乘数为 <u>1</u> 加被乘数
	1	0001	11 <u>1</u>	→1，
	0	1000	111 <u>1</u>	无乘数位，结束

改进的笔算乘法小结

- 两个 n 位数乘法运算可用 加 n 次和移位 n 次 实现
- 由乘数的末位决定 被乘数是否与原部分积相加
- 乘数右移1位（末位舍弃），空出高位存放部分积的低位，
同时部分积 右移 1 位 形成新的部分积。
- 被乘数只与部分积的高 n 位相加
- 硬件需求：移位寄存器, 全加器

计算机的运算方法

- 算术移位与逻辑移位
- 定点运算
 - 加减法运算
 - 一位乘法
 - booth算法
 - 除法运算
- 浮点运算

原码一位乘运算规则

整数： 设 $[X]_{\text{原}} = \textcolor{red}{X}_n X_{n-1} \dots X_1 X_0$, $[Y]_{\text{原}} = \textcolor{red}{Y}_n Y_{n-1} \dots Y_1 Y_0$

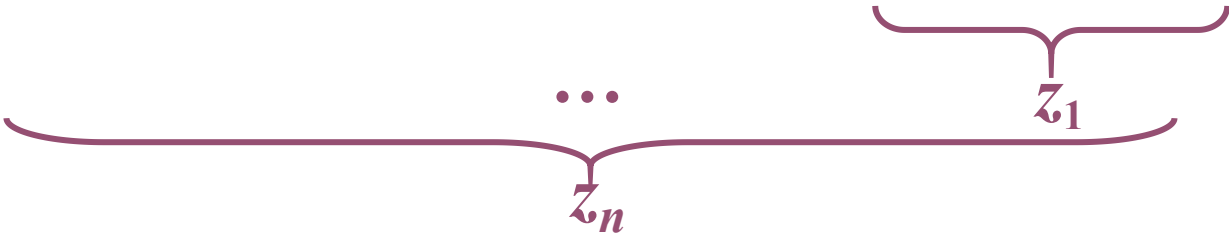
$$\begin{aligned}[X \times Y]_{\text{原}} &= (X_n \oplus Y_n)(\textcolor{blue}{0} X_{n-1} \dots X_1 X_0 \times \textcolor{blue}{0} Y_{n-1} \dots Y_1 Y_0) \\ &= (X_n \oplus Y_n)(|X| \times |Y|)\end{aligned}$$

小数： 设 $[x]_{\text{原}} = \textcolor{red}{x}_0 . x_1 x_2 \dots x_n$, $[y]_{\text{原}} = \textcolor{red}{y}_0 . y_1 y_2 \dots y_n$

$$\begin{aligned}[x \times y]_{\text{原}} &= (x_0 \oplus y_0) . (0 . x_1 x_2 \dots x_n \times 0 . y_1 y_2 \dots y_n) \\ &= (x_0 \oplus y_0) . (|x| \times |y|)\end{aligned}$$

乘积的符号位单独处理, 数值部分为绝对值相乘

小数原码一位乘递推公式(选学)

$$\begin{aligned} |x| \times |y| &= |x|(0.y_1y_2 \dots y_n) \\ &= |x| \times (y_1 \times 2^{-1} + y_2 \times 2^{-2} + \dots + y_n \times 2^{-n}) \\ &= 2^{-1}(y_1 \times |x| + 2^{-1}y_2 \times |x| + \dots + 2^{-n+1}y_n \times |x|) \\ &= 2^{-1}(y_1 \times |x| + 2^{-1}(y_2 \times |x| + \dots + 2^{-1}(y_n \times |x| + \underbrace{0}_{z_0}) \dots)) \end{aligned}$$


$$z_0 = 0$$

$$z_1 = 2^{-1}(y_n|x| + z_0)$$

$$z_2 = 2^{-1}(y_{n-1}|x| + z_1)$$

...

$$z_i = 2^{-1}(y_{n+1-i}|x| + z_{i-1})$$

...

$$z_n = 2^{-1}(y_1|x| + z_{n-1})$$

$x = -0.1110, y = 0.1101$, 求 $[x \times y]_{\text{原}}$ (选学)

数值部分的运算

部分积(乘数暂存)	丢弃	说明
0.0000	110 <u>1</u>	部分积，初态 $z_0 = 0$
+ 0.1110		乘数为 <u>1</u> ，加 $ x $
0.1110	110 <u>1</u>	逻辑右移→1，
0.0111	0 <u>110</u> 1	得 z_1
+ 0.0000		乘数为 <u>0</u> ，加 0
0.0111	0 <u>110</u>	逻辑右移→1，
0.0011	1 <u>011</u> 01	得 z_2
+ 0.1110		乘数为 <u>1</u> ，加 $ x $
1.0001	1 <u>011</u>	逻辑右移→1，
0.1000	1 <u>101</u> 101	得 z_3
+ 0.1110		乘数为 <u>1</u> ，加 $ x $
1.0110	1 <u>101</u>	逻辑右移→1，
0.1011	0 <u>110</u> 1101	得 z_4 ，结束

整数原码一位乘递推公式

$$\begin{aligned}
 |X| \times |Y| &= |X| (Y_{n-1} \dots Y_1 Y_0) = 2^{n-1} Y_{n-1} |X| + 2^{n-2} Y_{n-2} |X| + \dots + 2^1 Y_1 |X| + 2^0 Y_0 |X| \\
 &= 2^n * (2^{-1} Y_{n-1} |X| + 2^{-2} Y_{n-2} |X| + \dots + 2^{-n+1} Y_1 |X| + 2^{-n} Y_0 |X|) \\
 &= 2^n * (2^{-1} * (Y_{n-1} |X| + 2^{-1} Y_{n-2} |X| + \dots + 2^{-n+2} Y_1 |X| + 2^{-n+1} Y_0 |X|)) \\
 &= 2^n * (2^{-1} * (Y_{n-1} |X| + 2^{-1} (Y_{n-2} |X| + \dots + 2^{-n+3} Y_1 |X| + 2^{-n+2} Y_0 |X|))) \\
 &\quad \dots\dots\dots \dots\dots\dots \dots\dots\dots \dots\dots\dots \dots\dots\dots \dots\dots\dots \\
 &= 2^n * 2^{-1} * \underbrace{(Y_{n-1} |X| + 2^{-1} (Y_{n-2} |X| + 2^{-1} (\dots + 2^{-1} (Y_1 |X| + 2^{-1} (Y_0 |X| + 0))))}_{Z_n} \dots) \underbrace{\quad\quad\quad}_{Z_0}
 \end{aligned}$$

$$Z_0 = 0$$

$$Z_1 = 2^{-1} (Y_0 |X| + Z_0)$$

$$Z_2 = 2^{-1} (Y_1 |X| + Z_1)$$

...

$$Z_n = 2^{-1} (Y_{n-1} |X| + Z_{n-1})$$

$2^n?$

$X = -1110$, $Y = 1101$, 求 $[X \times Y]_{\text{原}}$

数值部分的运算	积	乘数	丢弃	说明
	0 0 0 0 0 0 0 0	<u>1 1 0 1</u>		高部分积置0, $Z_0 = 0$
+ 1 1 1 0	1 1 1 0 0 0 0 0			乘数为 <u>1</u> , 加 $ X $
	0 1 1 1 0 0 0 0	0 <u>1 1 0</u>	1	逻辑右移 $\rightarrow 1$, 得 Z_1
+ 0 0 0 0	0 1 1 1 0 0 0 0			乘数为 <u>0</u> , 加 0
	0 0 1 1 1 0 0 0	0 0 <u>1 1</u>	0 1	逻辑右移 $\rightarrow 1$, 得 Z_2
+ 1 1 1 0	1 0 0 0 1 1 0 0			乘数为 <u>1</u> , 加 $ X $
	0 1 0 0 0 1 1 0	0 0 0 <u>1</u>	1 0 1	逻辑右移 $\rightarrow 1$, 得 Z_3
+ 1 1 1 0	1 0 1 1 0 1 1 0			乘数为 <u>1</u> , 加 $ X $
	0 1 0 1 1 0 1 1	0 0 0 0	1 1 0 1	逻辑右移 $\rightarrow 1$, 得 Z_4 , 结束

$$X = -1110, Y = 1101, \text{求}[X \times Y]_{\text{原}}$$

数值部分的运算
(优化版本)

部分积(乘数暂存)	丢弃	说明
0 0 0 0	1 1 0 1	高部分积置0, $Z_0 = 0$
+ 1 1 1 0		乘数为 1, 加 $ X $
1 1 1 0	1 1 0 1	逻辑右移 $\rightarrow 1$,
0 1 1 1	0 1 1 0	得 Z_1
+ 0 0 0 0		乘数为 0, 加 0
0 1 1 1	0 1 1 0	逻辑右移 $\rightarrow 1$,
0 0 1 1	1 0 1 1	得 Z_2
+ 1 1 1 0		乘数为 1, 加 $ X $
1 0 0 0 1	1 0 1 1	逻辑右移 $\rightarrow 1$,
0 1 0 0 0	1 1 0 1	得 Z_3
+ 1 1 1 0		乘数为 1, 加 $ X $
1 0 1 1 0	1 1 0 1	逻辑右移 $\rightarrow 1$,
0 1 0 1 1	0 1 1 0	得 Z_4 , 结束

原码乘法小结

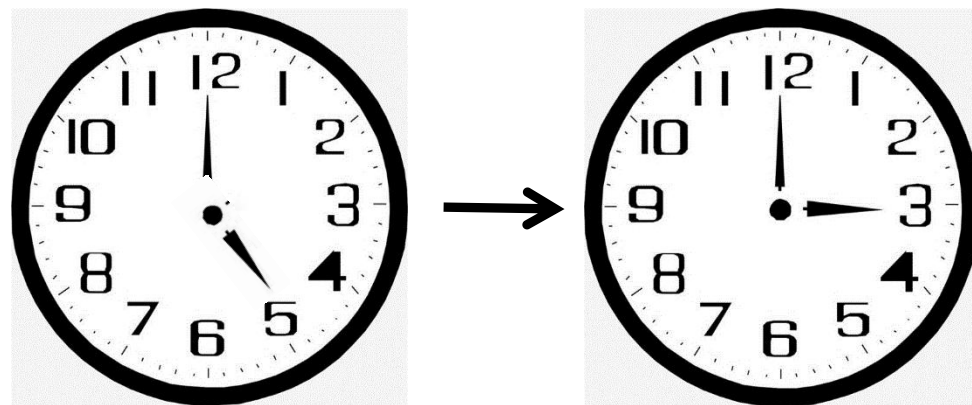
- 乘积的符号位 = 被乘数的符号位 $\oplus$ 乘数的符号位
- 数值部分按绝对值相乘

特点：

- 绝对值运算
- 用移位的次数判断乘法是否结束
- 逻辑移位

(回顾) 补码的定义

- 逆时针: $5 - 2 = 3$
- 顺时针: $5 + 10 = 3 + 12$
- 时钟以12为模
- 可见 -2 可用 $+10$ 代替
 - 称 $+10$ 是 -2 (以 12 为模) 的补数
 - 记作 $-2 \equiv +10 \pmod{12}$



减法 $\longrightarrow$ 加法

对于n位数值位的二进制数 x

$$[x]_{\text{补}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

补码乘法

- 如果机器字长 N 位，真值 X 的补码 $[X]_{\text{补}} = 2^N + X$
- $[X]_{\text{补}}$ 与真值 X 在模 2^N 下同余， $[X]_{\text{补}} \equiv X \pmod{2^N}$
- 补码运算属于**模运算**体系（密码学、哈希算法的理论基础）
- 同余定理（部分）
 - 对称性：若 $a \equiv b \pmod{m}$ ，则 $b \equiv a \pmod{m}$
 - 若 $X \equiv b \pmod{m}$ ， $Y \equiv d \pmod{m}$ ，则以下性质成立：
 - 加法： $X + Y \equiv b + d \pmod{m} \longrightarrow X + Y \equiv X_{\text{补}} + Y_{\text{补}}$
 - 减法： $X - Y \equiv b - d \pmod{m} \longrightarrow X - Y \equiv X_{\text{补}} - Y_{\text{补}}$
 - 乘法： $X \times Y \equiv b \times d \pmod{m} \longrightarrow X \times Y \equiv X_{\text{补}} \times Y_{\text{补}}$

补码一位乘法运算规则（小数）（选学）

设被乘数 $[x]_{\text{补}} = x_0.x_1x_2 \dots x_n$ ，乘数 $[y]_{\text{补}} = y_0.y_1y_2 \dots y_n$

●被乘数任意，乘数为正（类似原码乘）

但加和移位按补码规则运算，积的符号自然形成

●被乘数任意，乘数为负

乘数 $[y]_{\text{补}}$ ，去掉符号位，操作同上，最后加 $[-x]_{\text{补}}$ ，校正

$$\begin{aligned}[x]_{\text{补}} \times [y]_{\text{补}} &= [x]_{\text{补}} \times (1.0_1 \dots 0_n + 0.y_1y_2 \dots y_n)_{\text{补}} \\ &= [x]_{\text{补}} \times [0.y_1y_2 \dots y_n]_{\text{补}} - [x]_{\text{补}} \\ &= [x]_{\text{补}} \times [0.y_1y_2 \dots y_n]_{\text{补}} + [-x]_{\text{补}}\end{aligned}$$

补码一位乘法运算规则（整数）

设 被乘数 $[X]_{\text{补}} = X_n \dots X_1 X_0$, 乘数 $[Y]_{\text{补}} = Y_n \dots Y_1 Y_0$

- 被乘数任意, 乘数为正

- 类似原码乘, 加和移位按补码规则, 积的符号自然形成

- 被乘数任意, 乘数为负

- 乘数 $[Y]_{\text{补}}$, 去掉符号位, 其他操作同上, 最后加 $2^n[-X]_{\text{补}}$ (校正)

$$\begin{aligned}[X]_{\text{补}} \times [Y]_{\text{补}} &= [X]_{\text{补}} \times [10_1 \dots 0_n + 0Y_{n-1} \dots Y_1 Y_0]_{\text{补}} \\ &= [X]_{\text{补}} \times [0Y_{n-1} \dots Y_1 Y_0]_{\text{补}} + (-2^n)[X]_{\text{补}} \\ &= [X]_{\text{补}} \times [0Y_{n-1} \dots Y_1 Y_0]_{\text{补}} + 2^n[-X]_{\text{补}}\end{aligned}$$

计算机的运算方法

- 算术移位与逻辑移位
- 定点运算
 - 加减法运算
 - 一位乘法
 - booth算法
 - 除法运算
- 浮点运算

Booth算法(小数, 被乘数、乘数符号任意)(选学)

设 x 和 y 的补码表示分别为 $[x]_{\text{补}} = x_0.x_1x_2 \dots x_n$, $[y]_{\text{补}} = y_0.y_1y_2 \dots y_n$ 。

$$\begin{aligned} [x \times y]_{\text{补}} &= [x]_{\text{补}} [0.y_1 \dots y_n + y_0.0_10_2 \dots 0_n]_{\text{补}} \\ &= [x]_{\text{补}} (0.y_1 \dots y_n) + [x]_{\text{补}} \times [y_0.0_10_2 \dots 0_n]_{\text{补}} \\ &= [x]_{\text{补}} (y_1 2^{-1} + y_2 2^{-2} + \dots + y_n 2^{-n}) + [x]_{\text{补}} \times (-y_0) \\ &= [x]_{\text{补}} (-y_0 + y_1 2^{-1} + y_2 2^{-2} + \dots + y_n 2^{-n}) \quad \underline{\underline{2^{-i+1} - 2^{-i} = 2 * 2^{-i} - 2^{-i} = 2^{-i}}} \\ &= [x]_{\text{补}} [-y_0 + (y_1 2^0 - y_1 2^{-1}) + (y_2 2^{-1} - y_2 2^{-2}) + \dots + (y_n 2^{-(n-1)} - y_n 2^{-n})] \quad \text{附加位 } y_{n+1} = 0 \\ &= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + (y_3 - y_2) 2^{-2} + \dots + (y_n - y_{n-1}) 2^{-(n-1)} + (\underline{y_{n+1}} - \underline{y_n}) 2^{-n}] \\ &= (y_1 - y_0) [x]_{\text{补}} + \\ &\quad 2^{-1} \{ (y_2 - y_1) [x]_{\text{补}} + 2^{-1} \{ (y_3 - y_2) [x]_{\text{补}} + \dots + 2^{-1} \{ (\underline{y_{n+1}} - \underline{y_n}) [x]_{\text{补}} + \underline{0} \} \dots \} \} \end{aligned}$$

$\underline{z_1}$ $\underline{z_0} = 0$

Booth算法递推公式(选学)

$$[z_0]_{\text{补}} = 0 \quad y_{n+1} = 0$$

$$[z_1]_{\text{补}} = 2^{-1} \{ (y_{n+1} - y_n) [x]_{\text{补}} + [z_0]_{\text{补}} \}$$

$$[z_2]_{\text{补}} = 2^{-1} \{ (y_n - y_{n-1}) [x]_{\text{补}} + [z_1]_{\text{补}} \}$$

.....

$$[z_i]_{\text{补}} = 2^{-1} \{ (y_{n+2-i} - y_{n+1-i}) [x]_{\text{补}} + [z_{i-1}]_{\text{补}} \}$$

.....

$$[z_{n-i+1}]_{\text{补}} = 2^{-1} \{ (y_{i+1} - y_i) [x]_{\text{补}} + [z_{n-i}]_{\text{补}} \}$$

.....

$$[z_n]_{\text{补}} = 2^{-1} \{ (y_2 - y_1) [x]_{\text{补}} + [z_{n-1}]_{\text{补}} \}$$

$$[x \times y]_{\text{补}} = (y_1 - y_0) [x]_{\text{补}} + [z_n]_{\text{补}}$$

y_i	y_{i+1}	$y_{i+1} - y_i$	操作
0	0	0	$[z_{n-i}]_{\text{补}} \rightarrow 1$
0	1	1	$[z_{n-i}]_{\text{补}} + [x]_{\text{补}} \rightarrow 1$
1	0	-1	$[z_{n-i}]_{\text{补}} + [-x]_{\text{补}} \rightarrow 1$
1	1	0	$[z_{n-i}]_{\text{补}} \rightarrow 1$

算术右移

算术右移

最后一步不移位

$x=+0.0011, y=0.1011$, 用booth算法求 $[x \times y]_{\text{补}}$ (选学)

部分积(乘数暂存)	丢弃	说明
$\begin{array}{r} 0.0000 \\ + 1.1101 \\ \hline 1.1101 \\ 1.\underline{1}110 \end{array}$	$\begin{array}{r} 1.01010 \\ 10101 \\ 11010 \end{array}$	附加位 $y_{n+1}=y_5=0$ $y_4y_5=10$, 加 $[-x]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[z_1]_{\text{补}}$
$\begin{array}{r} + 0.0011 \\ 0.0001 \\ 0.\underline{0}000 \end{array}$	$\begin{array}{r} 11010 \\ 11101 \end{array}$	$y_3y_4=01$, 加 $[x]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[z_2]_{\text{补}}$
$\begin{array}{r} + 1.1101 \\ 1.1101 \\ 1.\underline{1}110 \end{array}$	$\begin{array}{r} 11101 \\ 11110 \end{array}$	$y_2y_3=10$, 加 $[-x]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[z_3]_{\text{补}}$
$\begin{array}{r} + 0.0011 \\ 0.0001 \\ 0.\underline{0}000 \end{array}$	$\begin{array}{r} 11110 \\ 11111 \end{array}$	$y_1y_2=01$, 加 $[x]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[z_4]_{\text{补}}$
$\begin{array}{r} + 1.1101 \\ 1.1101 \end{array}$	1111	$y_0y_1=10$, 加 $[-x]_{\text{补}}$ 结束(最后一步不移位)

$$[x]_{\text{补}} = 0.0011$$

$$[y]_{\text{补}} = 1.0101$$

$$[-x]_{\text{补}} = 1.1101$$

$y_i \ y_{i+1}$	$y_{i+1}-y_i$	操作
0 0	0	$\rightarrow 1$
0 1	1	$+ [x]_{\text{补}} \rightarrow 1$
1 0	-1	$+ [-x]_{\text{补}} \rightarrow 1$
1 1	0	$\rightarrow 1$

$$\therefore [x \times y]_{\text{补}} = 1.11011111$$

Booth算法（整数，被乘数、乘数符号任意）

设 X 和 Y 的补码表示分别为 $[X]_{\text{补}} = X_n, X_{n-1} \dots X_0$, $[Y]_{\text{补}} = Y_n, Y_{n-1} \dots Y_0$ 。

$$\begin{aligned}
 [X \times Y]_{\text{补}} &= [X]_{\text{补}} [0Y_{n-1} \dots Y_0 + \textcolor{red}{Y_n 0_{n-1} 0_{n-2} \dots 0_0}]_{\text{补}} \\
 &= [X]_{\text{补}} (0Y_{n-1} \dots Y_0) + [X]_{\text{补}} \times [\textcolor{red}{Y_n 0_{n-1} 0_{n-2} \dots 0_0}]_{\text{补}} \\
 &= \textcolor{red}{2^n} * ([X]_{\text{补}} (Y_{n-1} 2^{-1} + Y_{n-2} 2^{-2} + \dots + Y_0 2^{-n}) + [X]_{\text{补}} \times \textcolor{red}{(-Y_n)}) \quad \text{2进制转换并提取} 2^n \\
 &= \textcolor{red}{2^n} * ([X]_{\text{补}} (-Y_n + \textcolor{red}{Y_{n-1} 2^{-1}} + \textcolor{blue}{Y_{n-2} 2^{-2}} + \dots + \textcolor{green}{Y_0 2^{-n}})) \quad \underline{\textcolor{red}{2^{-i+1} - 2^{-i} = 2 * 2^{-i} - 2^{-i} = 2^{-i}}} \\
 &= \textcolor{red}{2^n} * ([X]_{\text{补}} [\underbrace{-Y_n + (\textcolor{red}{Y_{n-1} 2^0 - Y_{n-1} 2^{-1}})}_{\text{}}] + (\textcolor{blue}{Y_{n-2} 2^{-1} - Y_{n-2} 2^{-2}}) + \dots + (\textcolor{green}{Y_0 2^{-(n-1)} - Y_0 2^{-n}})]) \\
 &= \textcolor{red}{2^n} * ([X]_{\text{补}} [(Y_{n-1} - Y_n) + (Y_{n-2} - Y_{n-1}) 2^{-1} + (Y_{n-3} - Y_{n-2}) 2^{-2} + \dots + (Y_0 - Y_1) 2^{-(n-1)} + \\
 &\quad (\textcolor{red}{Y_{-1}} - \textcolor{green}{Y_0}) 2^{-n}]) \quad \text{附加位 } \textcolor{red}{Y_{-1}} = 0 \\
 &= \textcolor{red}{2^n} * (Y_{n-1} - Y_n) [X]_{\text{补}} + 2^{-1} \{ \textcolor{red}{2^n} * (Y_{n-2} - Y_{n-1}) [X]_{\text{补}} + 2^{-1} \{ \textcolor{red}{2^n} * (Y_{n-3} - Y_{n-2}) [X]_{\text{补}} \dots + \\
 &\quad \underbrace{2^{-1} \{ \textcolor{red}{2^n} * (Y_{-1} - Y_0) [X]_{\text{补}} + \textcolor{brown}{0} \}}_{\text{}} \dots \} \} \\
 &\quad \textcolor{red}{[Z_1]_{\text{补}}} \quad \textcolor{red}{Z_0 = 0}
 \end{aligned}$$

Booth整数算法递推公式

$$[Z_0]_{\text{补}} = 0 \quad Y_{-1} = 0$$

$$[Z_1]_{\text{补}} = 2^{-1} \{ 2^n * (Y_{-1} - Y_0) [X]_{\text{补}} + [Z_0]_{\text{补}} \}$$

$$[Z_2]_{\text{补}} = 2^{-1} \{ 2^n * (Y_0 - Y_1) [X]_{\text{补}} + [Z_1]_{\text{补}} \}$$

.....

$$[Z_{i+1}]_{\text{补}} = 2^{-1} \{ 2^n * (Y_{i-1} - Y_i) [X]_{\text{补}} + [Z_i]_{\text{补}} \}$$

.....

$$[Z_n]_{\text{补}} = 2^{-1} \{ 2^n * (Y_{n-2} - Y_{n-1}) [X]_{\text{补}} + [Z_{n-1}]_{\text{补}} \}$$

$$[X \times Y]_{\text{补}} = 2^n * (Y_{n-1} - Y_n) [X]_{\text{补}} + [Z_n]_{\text{补}} \quad \text{最后一步不移位}$$

$Y_i Y_{i-1}$	$Y_{i-1} - Y_i$	操作
0 0	0	$[Z_i]_{\text{补}} \rightarrow 1$
0 1	1	$[Z_i]_{\text{补}} + 2^n * [X]_{\text{补}} \rightarrow 1$
1 0	-1	$[Z_i]_{\text{补}} + 2^n * [-X]_{\text{补}} \rightarrow 1$
1 1	0	$[Z_i]_{\text{补}} \rightarrow 1$

$X=+0011, Y=-1011$, 用booth算法求 $[X \times Y]_{\text{补}}$

部分积(乘数暂存)	丢弃	说明
$\begin{array}{r} 0, 0000 \\ + 1, 1101 \\ \hline 1, 1101 \\ 1, 1110 \end{array}$	$\begin{array}{r} 1, 0101 \\ 10101 \\ 11010 \end{array}$	附加位 $Y_{-1}=0, Z_0=0$ $Y_0Y_{-1}=10$, 加 $2^n \cdot [-X]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[Z_1]_{\text{补}}$
$\begin{array}{r} + 0, 0011 \\ 0, 0001 \\ 0, 0000 \end{array}$	$\begin{array}{r} 11010 \\ 11101 \end{array}$	$Y_1Y_0=01$, 加 $2^n \cdot [X]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[Z_2]_{\text{补}}$
$\begin{array}{r} + 1, 1101 \\ 1, 1101 \\ 1, 1110 \end{array}$	$\begin{array}{r} 11101 \\ 11110 \end{array}$	$Y_2Y_1=10$, 加 $2^n \cdot [-X]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[Z_3]_{\text{补}}$
$\begin{array}{r} + 0, 0011 \\ 0, 0001 \\ 0, 0000 \end{array}$	$\begin{array}{r} 11110 \\ 11111 \end{array}$	$Y_3Y_2=01$, 加 $2^n \cdot [X]_{\text{补}}$ 算术右移 $\rightarrow 1$ $[Z_4]_{\text{补}}$
$\begin{array}{r} + 1, 1101 \\ 1, 1101 \end{array}$	1111	$Y_4Y_3=10$, 加 $2^n \cdot [-X]_{\text{补}}$ 结束(最后一步不移位)

$[X]_{\text{补}} = 0,0011$
 $[Y]_{\text{补}} = 1,0101$
 $[-X]_{\text{补}} = 1,1101$

Y_iY_{i-1}	$Y_{i-1}-Y_i$	操作
0 0	0	$[Z_i]_{\text{补}} \rightarrow 1$
0 1	1	$[Z_i]_{\text{补}} + 2^n \cdot [X]_{\text{补}} \rightarrow 1$
1 0	-1	$[Z_i]_{\text{补}} + 2^n \cdot [-X]_{\text{补}} \rightarrow 1$
1 1	0	$[Z_i]_{\text{补}} \rightarrow 1$

$\therefore [X \times Y]_{\text{补}}$
 $= 1,11011111$

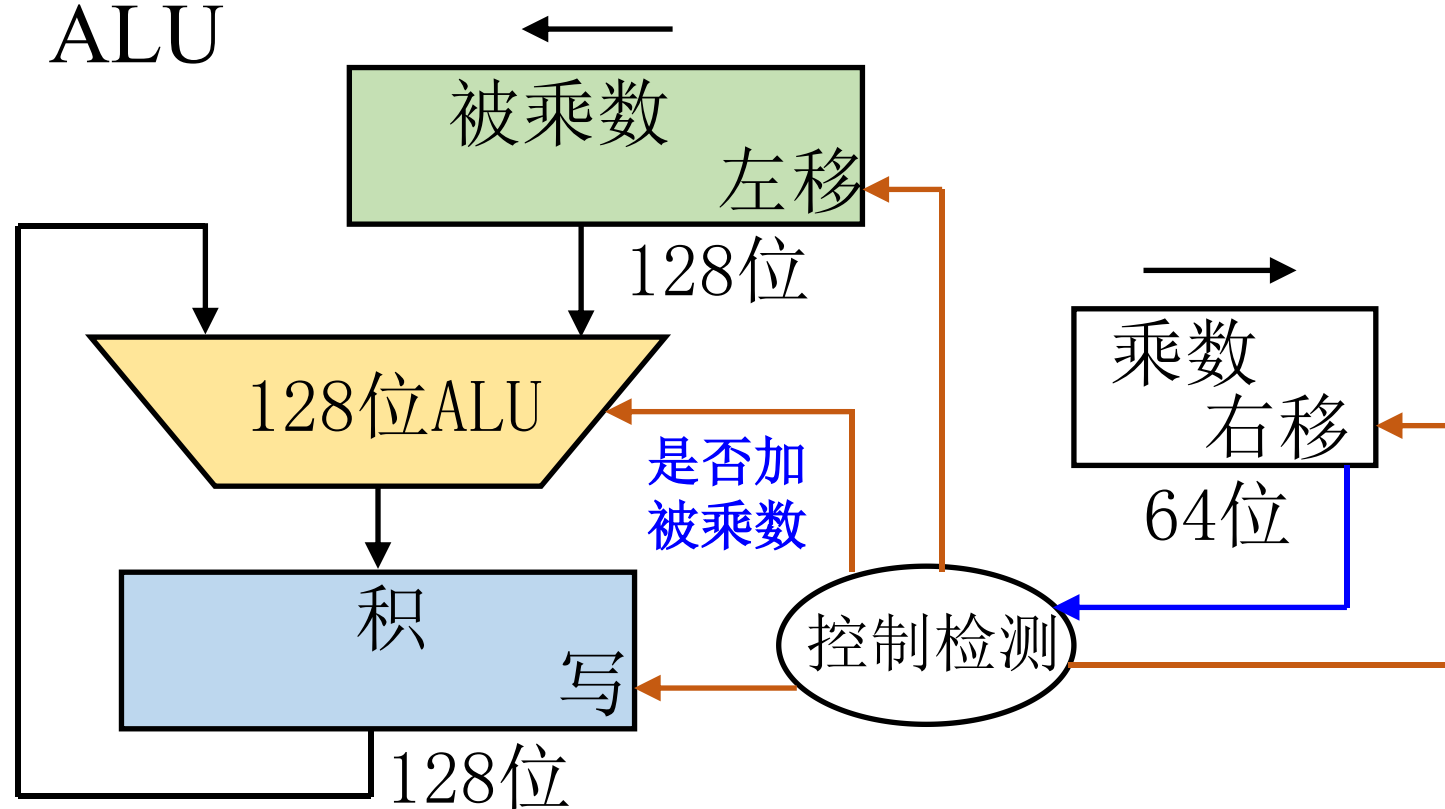
乘法小结

- 整数乘法与小数乘法基本相同
 - 用 逗号 代替小数点
 - 整数有整体移位操作，部分积最开始保存在积的高位
- 符号位：原码乘需要单独处理，补码乘 自然形成
- 原码乘去掉符号位运算, 即为无符号数乘法
- 不同的乘法运算需有不同的硬件支持

乘法器硬件示意图

- 被乘数寄存器 $2n$ 位
 - 被乘数 n 位，要左移一位 $n-1$ 次。
- 浪费：被乘数寄存器、ALU
 - 多数时间只用 n 位

$$\begin{array}{r} 1101 \\ \times 1011 \\ \hline 1101 \\ + 1101 \\ \hline 100111 \\ + 0000 \\ \hline 100111 \\ + 1101 \\ \hline 10001111 \end{array}$$



黑书图3-3

改良版乘法器硬件

A. 积寄存器129位(初值为: 65个0 和 64位的乘数)

- 最高位用于保存加法器的进位

B. 若乘数最右端为1

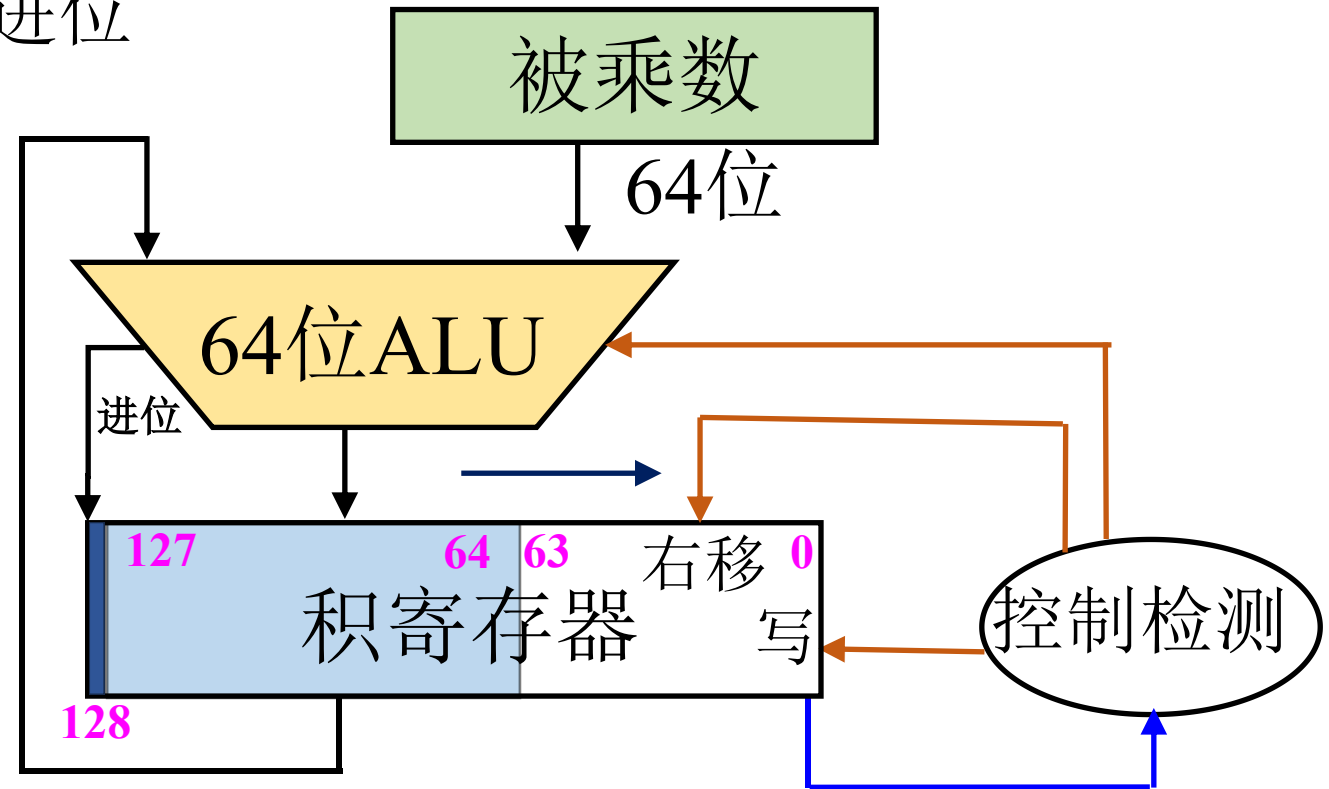
- 取积寄存器[127:64]
- 取出的值加上被乘数;
- **和**写入积寄存器[**128:64**]

C. 积寄存器整体右移一位

*B和C循环64次

*结果保存到积寄存器[127:0]

*快速乘法: 黑书3.3.3



黑书图3-5

计算机的运算方法

- 定点运算
 - 加减法运算
 - 一位乘法
 - booth算法
 - 除法运算
- 浮点运算

分析笔算除法（整数）

（数值位为4位） 设 $X = -101010$ $Y = 0011$ 求 $X \div Y$

$$\begin{array}{r} 0011 \\ 0011 \overline{) 101001} \\ \underline{-011} \\ 0100 \\ \underline{-0011} \\ 000101 \\ \underline{-000011} \\ 000001 \end{array}$$

$$X \div Y = -001101$$

余数 000001

/ 商符单独处理

? 心算上商

? 部分余数不动低位补“0”

? 如果部分余数大于除数
则减右移的除数

? 上商位置不固定

笔算除法的改进(适用于计算机)

笔算除法

商符单独处理

心算上商

部分余数不动低位补“0”

减右移一位的除数(绝对值)

上商位置 不固定

改进除法

符号位异或

$|x| - |y| > 0$ 上商 1

$|x| - |y| < 0$ 上商 0

部分余数左移1位低位补0

减除数(绝对值)

在商寄存器 最末位上商

原码除法

约定: 被除数和除数都不等于 0

整数定点除法 $X^* > Y^*$ (A^* 代表 $|A|$)

(小数定点除法 $x^* < y^*$)

$$[X]_{\text{原}} = X_n, X_{n-1} X_{n-2} \dots X_0$$

$$[Y]_{\text{原}} = Y_n, Y_{n-1} Y_{n-2} \dots Y_0$$

$$[X \div Y]_{\text{原}} = (X_n \oplus Y_n), (X^* \div Y^*)$$

商的符号位单独处理 $X_n \oplus Y_n$

数值部分为绝对值相除 $X^* \div Y^*$

恢复余数法

- 符号位单独处理，取除数和被除数**绝对值**进行运算；
 1. 余数（被除数）**减去** 除数 $[y^*]_{\text{补}}$ （+ $[-y^*]_{\text{补}}$ ）——>余数
 2. if 余数为负（不够减） then
 商上0，余数 $+ [y^*]_{\text{补}}$ （**恢复余数**）；
 else（够减）
 商上1
 3. if 循环次数<数据位数**n** then
 余数**逻辑左移**一位，加 $[-y^*]_{\text{补}}$ ；**重复**步骤2
 4. 结束

注：**余数的符号与被除数相同**

恢复余数法示例（小数，选学）

$x = -0.1011, y = -0.1101$, 求 $[x \div y]_{\text{原}}$

$[x]_{\text{原}} = 1.1011 \quad [y]_{\text{原}} = 1.1101 \quad [y^*]_{\text{补}} = 0.1101 \quad [-y^*]_{\text{补}} = 1.0011$

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$

②

被除数（余数）	商	说 明
$\begin{array}{r} 0.1011 \\ +1.0011 \\ \hline 1.1110 \\ +0.1101 \\ \hline \end{array}$	$\begin{array}{r} 0.0000 \\ \\ 0 \end{array}$	$+[-y^*]_{\text{补}}$ 余数为负，上商 0 恢复余数 $+ [y^*]_{\text{补}}$
$\begin{array}{r} 0.1011 \\ 1.0110 \\ +1.0011 \\ \hline 0.1001 \\ 1.0010 \\ +1.0011 \\ \hline \end{array}$	$\begin{array}{r} 0 \\ 0 \\ 01 \end{array}$	恢复后的余数 $\leftarrow 1$ 逻辑左移1位 $+[-y^*]_{\text{补}}$ 余数为正，上商 1 $\leftarrow 1$ 逻辑左移1位 $+[-y^*]_{\text{补}}$

恢复余数法示例——续（选学）

被除数（余数）	商	说 明
0.0101	01 1	余数为正， 上商 1
<u>0.1010</u>	011	<—1 逻辑左移1位
+1.0011		+ $[-y^*]_{\text{补}}$
1.1101	011 0	余数为负， 上商 0
+0.1101		恢复余数+ $[y^*]_{\text{补}}$
<u>0.1010</u>	0110	恢复后的余数
1.0100	0110	<—1 逻辑左移1位
+1.0011		+ $[-y^*]_{\text{补}}$
0.0111	0110 1	余数为正， 上商 1

$$x^*/y^* = 0.1101$$

$$\therefore [x/y]_{\text{原}} = 0.1101$$

余数为正:上商 1

余数为负:

上商 0, 恢复余数

- **上商 5 次**

- 第一次上商判**溢出**（约定：小数定点除法 $x^* < y^*$ ）

- **逻辑左移 4 次**

恢复余数法示例（整数）

$X = -1110$
 $Y = -0011$
求 $[X \div Y]_{\text{原}}$ 。
解：符号为
 $X_n \oplus Y_n$
 $= 1 \oplus 1 = 0$

$[X]_{\text{原}} = 1,1110$
 $[Y]_{\text{原}} = 1,0011$
 $[Y^*]_{\text{补}} = 0,0011$
 $[-Y^*]_{\text{补}} = 1,1101$

被除数（余数）	商	说 明
0,0000 1110 + 1,1101 1,1101 1110 + 0,0011 10,0000 1110	--- 0	1. $+[-Y^*]_{\text{补}}$ 2. 余数为负，上商0 3. 恢复余数 $+ [Y^*]_{\text{补}}$ 4. 恢复后的余数
0,0001 110 + 1,1101 1,1110 110 + 0,0011 10,0001 110	--00 --00 --00	5. 逻辑左移1位（第1次） 6. $+[-Y^*]_{\text{补}}$, 7. 余数为负，上商0 8. 恢复余数 $+ [Y^*]_{\text{补}}$ 9. 恢复后的余数
0,0011 10 + 1,1101 10,0000 10	-000 001	10. 逻辑左移1位（第2次） 11. $+[-Y^*]_{\text{补}}$ 12. 余数为正，上商1
00,0001 0	001-	13. 逻辑左移1位（第3次）

恢复余数法示例（整数）——续

$X = -1110$	被除数（余数）	商	说 明
$Y = -0011$	<u>00</u> ,0001 0	001_	13. 逻辑左移1位(第3次)
求 $[X \div Y]_{\text{原}}$ 。	+ 1,1101		14. $+[-Y^*]_{\text{补}}$
	1,1110 0	001 0	15. 余数为负， 上商0
	+ 0,0011		16. 恢复余数 $+ [Y^*]_{\text{补}}$
$[X]_{\text{原}} = 1,1110$	10 ,0001 0	0010	17. 恢复后的余数
$[Y]_{\text{原}} = 1,0011$	<u>00</u> ,0010	010_	18. 逻辑左移1位(第4次)
$[Y^*]_{\text{补}} = 0,0011$	+ 1,1101		19. $+[-Y^*]_{\text{补}}$
$[-Y^*]_{\text{补}} = 1,1101$	1,1111	010 0	20. 余数为负， 上商0
	+ 0,0011		21. 恢复余数 $+ [Y^*]_{\text{补}}$
	10 ,0010	0100	22. 恢复后的余数

（余数与被除数同号）余数为-2， 商为4，

恢复余数法示例（整理）

$X = -1110$

$Y = -0011$

求 $[X \div Y]_{\text{原}}$ 。

解：符号为

$$X_n \oplus Y_n$$
$$= 1 \oplus 1 = 0$$

$[X]_{\text{原}} = 1,1110$

$[Y]_{\text{原}} = 1,0011$

$[Y^*]_{\text{补}} = 0,0011$

$[-Y^*]_{\text{补}} = 1,1101$

被除数（余数）	商	说 明
0,0000 1110 + 1,1101 1,1101 1110	0	1. $+[-Y^*]_{\text{补}}$ 2. 余数 R_1 为负，上商0
+ 0,0011 10,0000 1110 0,0001 110 + 1,1101 1,1110 110	__00 __00	3. 恢复余数 4. 恢复后的余数 $R_1 + [Y^*]_{\text{补}}$ 5. 逻辑左移1位 $2(R_1 + [Y^*]_{\text{补}})$ 6. $+[-Y^*]_{\text{补}}$ 7. 余数 $R_2 = 2R_1 + [Y^*]_{\text{补}}$ 为负，商0
+ 0,0011 10,0001 110 0,0011 10 + 1,1101 10,0000 10	__00 __000 001	8. 恢复余数 $+ [Y^*]_{\text{补}}$ 9. 恢复后的余数 10. 逻辑左移1位 $2(R_2 + [Y^*]_{\text{补}})$ 11. $+[-Y^*]_{\text{补}}$ 12. 余数 $R_3 = 2R_2 + [Y^*]_{\text{补}}$ 为正，商1
0,0001 0	001_	13. 逻辑左移1位 ($2R_3$)

恢复余数法示例（整理）——续

$X = -1110$	被除数（余数）	商	说 明
$Y = -0011$	<u>00,0001 0</u>	001_	13. 逻辑左移1位($2R_3$)
求 $[X \div Y]_{\text{原}}$ 。	+ 1,1101		14. $+[-Y^*]_{\text{补}}$
	1,1110 0	0010	15. 余数 $R_4 = 2R_3 + [Y^*]_{\text{补}}$ 为负，商0
$[X]_{\text{原}} = 1,1110$	+ 0,0011		16. 恢复余数 $+ [Y^*]_{\text{补}}$
	1 0,0001 0	0010	17. 恢复后的余数
$[Y]_{\text{原}} = 1,0011$	00,0010	010_	18. 逻辑左移1位 $2(R_4 + [Y^*]_{\text{补}})$
$[Y^*]_{\text{补}} = 0,0011$	+ 1,1101		19. $+[-Y^*]_{\text{补}}$
$[-Y^*]_{\text{补}} = 1,1101$	1,1111	0100	20. 余数 $R_5 = 2R_4 + [Y^*]_{\text{补}}$ 为负，商0
	+ 0,0011		21. 恢复余数 $+ [Y^*]_{\text{补}}$
	1 0,0010	0100	22. 恢复后的余数

不恢复余数法（加减交替法）

●恢复余数法运算规则变换

余数 $R_i > 0$ ，上商 1，逻辑左移1位， $+[-y^*]_{\text{补}}$ ； $(2R_i + [-y^*]_{\text{补}})$

余数 $R_i < 0$ ，上商0， $\underline{+[y^*]_{\text{补}}}$ 恢复余数，逻辑左移1位， $+[-y^*]_{\text{补}}$ 。

$$2(R_i + [y^*]_{\text{补}}) + [-y^*]_{\text{补}} = 2R_i + [y^*]_{\text{补}}$$

●不恢复余数法运算规则

- 余数为正，上商1，逻辑左移1位， $+[-y^*]_{\text{补}}$

- 余数为负，上商0，逻辑左移1位， $+ [y^*]_{\text{补}}$

- 加减交替

不恢复余数法示例（整数）

$$X = -1110$$

$$Y = -0011$$

求 $[X \div Y]_{\text{原}}$ 。

解：符号为

$$\begin{aligned} X_n \oplus Y_n \\ = 1 \oplus 1 = 0 \end{aligned}$$

$$[X]_{\text{原}} = 1,1110$$

$$[Y]_{\text{原}} = 1,0011$$

$$[Y^*]_{\text{补}} = 0,0011$$

$$[-Y^*]_{\text{补}} = 1,1101$$

被除数（余数）	商	说 明
0,0000 1110 + 1,1101		1. $+[-Y^*]_{\text{补}}$
1,1101 1110	<u> </u> <u> </u> <u> </u> 0	2. 余数 R_1 为负， 上商0
1,1011 110 + 0,0011	<u> </u> <u> </u> <u> </u> <u> </u>	3. 逻辑左移1位 $2R_1$
1,1110 110	<u> </u> <u> </u> <u> </u> 00	4. $+ [Y^*]_{\text{补}}$
1,1101 10 + 0,0011	<u> </u> <u> </u> <u> </u> <u> </u>	5. 余数 $R_2=2R_1+[Y^*]_{\text{补}}$ 为负， 商0
10,0000 10	<u> </u> <u> </u> <u> </u> <u> </u>	6. 逻辑左移1位 $2R_2$
0,0001 0 + 1,1101	<u> </u> <u> </u> <u> </u> <u> </u>	7. $+ [Y^*]_{\text{补}}$
1,1110 0	<u> </u> <u> </u> <u> </u> 001	8. 余数 $R_3=2R_2+[Y^*]_{\text{补}}$ 为正， 商1
1,1100 + 0,0011	001 <u> </u>	9. 逻辑左移1位 $2R_3$
1,1111 + 0,0011	001 0	10. $+ [-Y^*]_{\text{补}}$
10,0010	010 <u> </u>	11. 余数 $R_4=2R_3+[Y^*]_{\text{补}}$ 为负， 商0
		12. 逻辑左移1位 $2R_4$
	010 0	13. $+ [Y^*]_{\text{补}}$
		14. 余数 $R_5=2R_4+[Y^*]_{\text{补}}$ 为负， 商0
	0100	15. 恢复余数 $+ [Y^*]_{\text{补}}$

恢复余数法（运算规则验证）

• $x = -0.1011$, $y = -0.1101$, 求 $\left[\frac{x}{y}\right]_{\text{原}}$

$[x]_{\text{原}} = 1.1011$ $[y]_{\text{原}} = 1.1101$ $[y^*]_{\text{补}} = 0.1101$ $[-y^*]_{\text{补}} = 1.0011$

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$

②被除数（余数）	商	说 明
$\boxed{0.1011}$ $+1.0011$	0.0000	$+[-y^*]_{\text{补}}$
1.1110 $+0.1101$	0	$R_1 < 0$ 余数为负，上商0 恢复余数 $+ [y^*]_{\text{补}}$
$\boxed{0.1011}$ 1.0110 $+1.0011$	0	恢复后的余数 $(R_1 + y^*)$ $\leftarrow 1$ 逻辑左移 $(2(R_1 + y^*))$ $+ [-y^*]_{\text{补}}$
0.1001 1.0010 $+1.0011$	01	$R_2 = 2R_1 + y^*$, 余数为正，上商1
1.0010 $+1.0011$	01	$\leftarrow 1$ 逻辑左移 $2R_2$ $+ [-y^*]_{\text{补}}$

被除数（余数）	商	说 明
0 . 0 1 0 1	0 1 1	$R_3 = 2R_2 - y^*$ 余数为正，上商 1
<u>0 . 1 0 1 0</u> +1 . 0 0 1 1	0 1 1	$\leftarrow -1$ 逻辑左移 $+[-y^*]_{\text{补}}$
1 . 1 1 0 1	0 1 1 0	$R_4 = 2R_3 - y^*$ 余数为负，上商 0
+0 . 1 1 0 1		恢复余数 $+ [y^*]_{\text{补}}$
<u>0 . 1 0 1 0</u>	0 1 1 0	恢复后的余数 $R_4 + y^*$
1 . 0 1 0 0	0 1 1 0	$\leftarrow -1$ 逻辑左移 $2(R_4 + y^*)$
+1 . 0 0 1 1		$+ [-y^*]_{\text{补}}$
0 . 0 1 1 1	0 1 1 0 1	$R_5 = 2R_4 + y^*$ 余数为正，上商 1

$$\frac{x^*}{y^*} = 0.1101$$

上商 **5** 次

$$\therefore \left[\frac{x}{y} \right]_{\text{原}} = 0.1101$$

第一次上商**判溢出**（约定：小数定点除法 $x^* < y^*$ ）

逻辑移位 **4** 次

例. $x = -0.1011$, $y = -0.1101$, 求 $[x/y]_{\text{原}}$

解: 0.1011	0.0000		$[x]_{\text{原}} = 1.1011$
$+1.0011$		$+[-y^*]_{\text{补}}$	
1.1110	0	余数为负, 上商 0	$[y]_{\text{原}} = 1.1101$
1.1100	0	$\leftarrow 1$ 逻辑左移	
$+0.1101$		$+ [y^*]_{\text{补}}$	$[x^*]_{\text{补}} = 0.1011$
0.1001	01	余数为正, 上商 1	$[y^*]_{\text{补}} = 0.1101$
1.0010	01	$\leftarrow 1$ 逻辑左移	$[-y^*]_{\text{补}} = 1.0011$
$+1.0011$		$+ [-y^*]_{\text{补}}$	
0.0101	011	余数为正, 上商 1	
0.1010	011	$\leftarrow 1$ 逻辑左移	
$+1.0011$		$+ [-y^*]_{\text{补}}$	
1.1101	0110	余数为负, 上商 0	
1.1010	0110	$\leftarrow 1$ 逻辑左移	
$+0.1101$		$+ [y^*]_{\text{补}}$	
0.0111	01101	余数为正, 上商 1	

不恢复余数法（加减交替法）

●运算规则

- 余数为正，上商1，逻辑左移1位， $+[-y^*]_{\text{补}}$
- 余数为负，上商0，逻辑左移1位， $+ [y^*]_{\text{补}}$

●特点：

- 移位 n 次
- 加 $n+1$ 次
- 加减交替
- 用移位的次数判断除法是否结束